Fundação Universidade Federal do ABC
Pró-reitoria de Pesquisa
Av. dos Estados, 5001, Santa Terezinha, Santo André/SP, CEP 09210-580
Bloco L, 3ºAndar, Fone (11) 3356-7617
iniciacao@ufabc.edu.br

Relatório Parcial de Bolsa de Iniciação Científica (PIBIC) do CNPq, referente ao Edital: 01/2025

Nome do aluno: Eduardo do Amaral Silva e Biella

Assinatura do aluno:

Nome do orientador: Prof. Dr. Aritanan Borges Garcia Gruber

Assinatura do orientador:

Título do projeto: Aprendizado por Reforço para Problemas de Otimização Combinatória

Palavras-chave do projeto: Otimização Combinatória, Processos de Decisão de Markov, Aprendizado por Reforço Profundo, Aproximação de Funções, Estabilidade de Convergência.

Área do conhecimento do projeto: Ciência da Computação, Matemática

Bolsista: Sim: PIBIC

Santo André

2025

Resumo

Esta pesquisa explora a fundamentação teórica da aplicação do Aprendizado por Reforço (*Reinforcement Learning*, RL) para a resolução de problemas de Otimização Combinatória (*Combinatorial Optimization*, CO) ([2, 5, 8]), com foco no Problema do Caixeiro Viajante (*Travelling Salesman Problem*, TSP).

O TSP, um problema canônico da classe NP-difícil ([1, 4]), serve como um modelo fundamental para aplicações práticas em diversas outras áreas. A abordagem central deste projeto consiste em formalizar o TSP como um Processo de Decisão de Markov (*Markov Decision Process*, MDP) ([7]), permitindo que um agente de RL aprenda, de forma autônoma, uma política para construir rotas semi-ótimas. Um desafio inevitável desta formulação é a “maldição da dimensionalidade”, onde o espaço de estados cresce exponencialmente com o número de cidades a serem visitadas, tornando inviáveis os métodos tabulares tradicionais.

Para superar esta barreira, neste projeto investiga-se a necessidade de aproximações de funções, com o uso de redes neurais profundas. Serão analisadas metodologias de Aprendizado por Reforço Profundo, como o *Deep Q-Networks* (DQN) e os algoritmos de Gradiente de Política (*REINFORCE*) e arquiteturas especializadas para dados estruturados, como o Redes Neurais de Grafos (*Graph Neural Networks*, GNNs) ([3]) e o Redes de Ponteiros (*Pointer Networks*, PN).

Por fim, este estudo aprofunda-se na análise da convergência e estabilidade destes métodos aproximados. Diferente da Programação Dinâmica, que garante otimalidade pelo Teorema do Ponto Fixo de Banach, o uso de aproximações de funções não lineares gera instabilidade, pois a propriedade de contração do Operador de Bellman é perdida. O projeto examina como mecanismos como *Target Networks* mitigam essa instabilidade e utiliza a teoria de Aproximação Estocástica, especialmente o método da Equação Diferencial Ordinário (ODE), para justificar a convergência dos algoritmos de aprendizado iterativo para ótimos locais ou pontos de equilíbrio estáveis. ([6])

Sumário

1. Introdução....4
1.1. O Problema do Caixeiro Viajante (TSP)....4
1.2. Justificativa e Motivação.... 5
1.3. A Abordagem Data-Driven e a Escassez de Rótulos....6
1.4. Objetivos.... 7
1.4.1. Objetivo Geral.... 7
1.4.2. Objetivos Específicos....7
2. Fundamentação Teórica.... 9
2.1. Aprendizado por Reforço.... 9
2.2. Processos de Decisão de Markov (MDP) e SSP.... 11
2.2.1. Políticas e Funções de Valor Ótimas....13
2.3. Modelando o TSP como um MDP....14
3. A Necessidade de Aproximação....16
3.1. A Maldição da Dimensionalidade....16
3.2. O Operador de Projeção.... 17
3.3. Viés Indutivo e Representação de Grafos....18
3.3.1. A Natureza Não-Euclidiana dos Dados....18
3.3.2. Formalização de Simetrias....18
3.3.3. Graph Neural Networks (GNNs)....19
3.4. Arquiteturas Especializadas: Pointer Networks e Attention.... 19
3.4.1. Superando o Vocabulário Fixo.... 19
3.4.2. Mecanismo de Atenção....20
3.4.3. Garantia de Validez via Mascaramento (Masking)....20
4. Metodologia de Algoritmos de Aprendizado por Reforço Profundo.... 20
4.1. Métodos Baseados em Valor: Deep Q-Networks (DQN).... 22
4.1.1. O Mecanismo de Aprendizado por Diferença Temporal....23
4.1.2. Otimização do Gradiente Estocástico e o Erro TD....24
4.1.3. Mecanismos de Estabilidade Adicionais.... 25
4.2. Métodos Baseados em Política e sua Otimização....26
4.2.1. Gradiente de Política: REINFORCE....27
4.2.2. Métodos Ator-Crítico (Actor-Critic).... 29
5. Análise de Convergência e Estabilidade.... 31
5.1. Convergência em Métodos Tabulares....32
5.1.1. Propriedades de Contração e o Ponto Fixo de Banach.... 32
5.1.2. Convergência sob Amostragem Estocástica....33
5.2. Aprendizado com Aproximação de Função.... 33
5.2.1. A geometria da Atualização.... 33
5.2.2. Instabilidade e Perda da Propriedade de Contração.... 34
5.2.3. A “Tríade Mortal” do Deep Q-Learning....34
5.3. Intuição via o Método da Equação Diferencial Ordinária (Ordinary Differential Equation ,ODE).... 35
5.3.1. Formalização da Atualização como Equação de Diferenças.... 35

5.3.2. A Trajetória Limite e o Campo Vetorial Médio..... 36
5.3.3. Condições de Robbins-Monro..... 37
6. Considerações Finais e Próximos Passos..... 39
6.1. Síntese..... 39
6.2. Cronograma Futuro..... 40
7. Referências Bibliográficas..... 41

1. Introdução

1.1. O Problema do Caixeiro Viajante (TSP)

A Otimização Combinatória (CO) constitui um ramo da ciência da computação e da matemática aplicada dedicado à busca de configurações ótimas em conjuntos finitos de soluções possíveis. Dentre os problemas estudados nesta área, o Problema do Caixeiro Viajante (Traveling Salesman Problem, TSP) destaca-se como um paradigma fundamental, tanto pela simplicidade de sua formulação quanto pela dificuldade intrínseca de sua resolução exata.

Formalmente, o TSP pode ser definido sobre um grafo completo ponderado G = (V, E), onde V representa um conjunto de N vértices (cidades) e E o conjunto de arestas que as conectam. A cada aresta (i, j) ∈ E associa-se um custo ou distância $c_{ij} > 0$. O objetivo é determinar uma permutação σ dos vértices que definam um Circuito Hamiltoniano (uma trajetória fechada que visita cada cidade exatamente uma vez e retorna ao ponto de origem) de modo que o custo total da rota seja minimizado:

$$\sum_{i=1}^{N-1} c_{\sigma(i),\,\sigma(i+1)} + c_{\sigma(N),\,\sigma(1)}$$

A relevância do TSP reside na sua complexidade computacional. A versão de decisão do problema (determinar se existe uma rota com custo menor ou igual a k) pertence à classe NP-completa. Consequentemente, a versão de otimização é classificada como NP-difícil (NP-*hard*), o que implica que não se conhece algoritmo capaz de resolvê-la de forma exata em tempo polinomial no pior caso.

A intratabilidade computacional do TSP torna-se evidente ao analisarmos o crescimento do espaço de busca. Uma abordagem ingênua de força bruta exigiria a avaliação de (N - 1)! / 2 permutações possíveis, um número que cresce fatorialmente e torna a computação inviável mesmo para instâncias não tão grandes. Métodos exatos e mais sofisticados, fundamentados na Programação Dinâmica, como o algoritmo de Held-Karp, conseguem reduzir essa complexidade para a ordem exponencial de $O(N^2\, 2^N)$. Contudo, mesmo essa redução é insuficiente para instâncias de grande escala; para um grafo com N=50 ou N=100 vértices, a exigência de memória e tempo de processamento para armazenar e computar a tabela de valores ótimos excede a capacidade dos sistemas computacionais modernos.

Esta barreira de complexidade, frequentemente denominada "maldição da dimensionalidade", impõe limites à aplicação de métodos exatos em cenários reais onde

decisões rápidas são necessárias. As pesquisas modernas visam superar as limitações das heurísticas estáticas através do Aprendizado de Máquina.

A relevância do TSP vai além da academia, fundamentando aplicações em diversas áreas, como em aplicações industriais, planejamento de rotas de veículos e fabricação de microchips. Diante da complexidade computacional, a pesquisa tem enfoque no desenvolvimento de algoritmos de aproximação e heurística que, embora não garantam a otimalidade, são capazes de encontrar soluções de alta qualidade em tempo hábil.

1.2. Justificativa e Motivação

A resolução de problemas de Otimização Combinatória, como o TSP, tem sido historicamente abordada sob dois paradigmas predominantes: métodos exatos e heurísticas manuais. As abordagens exatas, fundamentadas em Programação Linear Inteira ou Programação Dinâmica, oferecem garantias de otimalidade, mas sofrem com a barreira intransponível da complexidade computacional.

Em contrapartida, as heurísticas oferecem soluções tratáveis, porém frequentemente sub-ótimas e rígidas. O principal limitador destas abordagens reside no fato de serem projetadas manualmente por especialistas, baseando-se em regras fixas que não se adaptam às novas instâncias do sistema.

A mudança de paradigma em direção à Otimização Combinatória Neural e ao conceito de Learning to Optimize (Aprender a Otimizar) representa uma evolução significativa em relação aos métodos clássicos de planejamento e otimização, onde um algoritmo inicia a busca "do zero" para cada nova instância do grafo, o uso de Aprendizado por Reforço (RL) permite treinar um agente capaz de capturar regularidades na distribuição dos dados. O objetivo deixa de ser apenas resolver uma instância isolada, mas sim induzir uma política estocástica parametrizada que generalize a estratégia de construção de rotas para instâncias inéditas com baixo custo computacional.

Adicionalmente, a escolha pelo Aprendizado por Reforço, em detrimento do Aprendizado Supervisionado, é motivada por uma barreira prática fundamental: a escassez de rótulos ótimos. Treinar uma rede neural de forma supervisionada exigiria um vasto conjunto de dados contendo pares de grafos e suas respectivas soluções ótimas. Contudo, gerar tais rótulos para grafos grandes exige a execução de solucionadores exatos, o que recai no mesmo problema de intratabilidade que se deseja evitar. O RL supera essa limitação ao eliminar a necessidade de um "professor" externo; o agente aprimora sua política guiado exclusivamente pelo sinal de recompensa.

Por fim, este trabalho possui relevância teórica ao investigar a estabilidade destes métodos aproximados. A transição de tabelas de valores exatas para aproximadores de funções não-lineares (Redes Neurais Profundas) em algoritmos como o DQN introduz riscos de divergência conhecidos como a "Tríade Mortal". A análise crítica destes fenômenos e a comparação com métodos de Gradiente de Política (REINFORCE) contribuem para a compreensão das fronteiras entre a convergência teórica e a aplicabilidade prática de Deep RL em domínios combinatórios.

1.3. A Abordagem *Data-Driven* e a Escassez de Rótulos

A aplicação de técnicas de Aprendizado de Máquina à Otimização Combinatória poderia, intuitivamente, seguir o paradigma dominante do Aprendizado Supervisionado (SL). Nesta abordagem, uma rede neural seria treinada para imitar o comportamento de um oráculo, mapeando instâncias de grafos de entrada diretamente para suas soluções ótimas (rótulos). No entanto, tal metodologia encontra um obstáculo fundamental intrínseco à classe de complexidade do problema: a escassez e o custo computacional de dados rotulados.

Para treinar um modelo supervisionado capaz de generalizar, seria necessário um vasto conjunto de dados composto por milhões de pares (x,y), onde x representa a instância do problema (o grafo) e y a solução ótima (a permutação de custo mínimo). Contudo, para problemas NP-difíceis como o TSP, a geração de cada rótulo y exige a execução de solucionadores exatos.

Cria-se uma dependência cíclica indesejável: para treinar uma rede neural que resolva o problema eficientemente, precisaríamos primeiramente resolver o problema exaustivamente um número massivo de vezes, limitando a aplicação do método apenas a instâncias pequenas onde a solução exata já é tratável.

O Aprendizado por Reforço (RL) aparece como a solução metodológica para romper essa barreira, fundamentando o campo da Otimização Combinatória Neural. Ao contrário da abordagem supervisionada, o RL não requer um conjunto de treinamento com soluções ótimas pré-calculadas. Em vez disso, o agente atua em um ambiente onde a qualidade de suas decisões é avaliada a posteriori.

Nesta pesquisa, a abordagem data-driven via RL substitui a necessidade de um "professor" onisciente pelo uso da própria função objetivo do problema como sinal de aprendizado. O agente constrói soluções construtivamente e recebe uma recompensa. Através de algoritmos de otimização de política, a rede neural ajusta seus parâmetros para maximizar essa recompensa esperada. Isso permite que o sistema aprenda a otimizar instâncias de complexidade onde a solução ótima é desconhecida, guiando-se apenas

pelas propriedades geométricas e topológicas dos grafos observados durante o treinamento. Esta capacidade de "aprender fazendo" é o que viabiliza a resolução para instâncias de maior porte, onde a supervisão exata é computacionalmente inviável.

1.4. Objetivos

1.4.1. Objetivo Geral

O objetivo desta pesquisa é investigar a fundamentação teórica da aplicação de Aprendizado por Reforço ao Problema do Caixeiro Viajante (TSP). O estudo foca na transição metodológica de abordagens de planejamento exato (Programação Dinâmica tabular) para o paradigma de “Aprender a Otimizar” (*Learning to Optimize*), onde aproximações de funções não-lineares são empregadas para superar a intratabilidade computacional. A pesquisa visa analisar as condições de estabilidade e convergência contraste entre as garantias de otimalidade providas pelos métodos de Deep Learning (DQN e REINFORCE) necessários para tratar a complexidade combinatória do problema.

Um ponto crucial na investigação é a análise das condições de estabilidade e convergência, examinando como a introdução de arquiteturas de redes neurais altera as garantias de contração dos operadores de Bellman e como algoritmos baseados em política (como REINFORCE) se comparam aos métodos baseados em valor (como DQN) em domínios combinatórios.

1.4.2. Objetivos Específicos

Para o desenvolvimento desta pesquisa, nesta fase inicial, serão abordadas as seguintes metas específicas, estruturadas para conduzir a pesquisa da fundamentação matemática à validação empírica:

1. Modelar matematicamente o Problema do Caixeiro Viajante (TSP) não apenas como uma otimização combinatória estática, mas como um Problema de Caminho Mais Curto Estocástico (Stochastic Shortest Path, SSP) ou Processo de Decisão de Markov (MDP) episódico. O objetivo é estabelecer a equivalência entre a busca do custo mínimo em grafos determinísticos e a otimização de políticas em ambientes estocásticos, explicando a formulação da recompensa usada na aplicação de agentes de aprendizado por Reforço, que lidam com a Exploração e a Explotação.

2. Compreensão dos desafios computacionais, com a “maldição da dimensionalidade” ([7, 8]), gerado pelo crescimento fatorial do espaço de estados, que inviabiliza a aplicação de métodos tabulares e exatos (como o algoritmo de Held-Karp) para instâncias de larga escala. Isso justifica teoricamente a necessidade de generalização via Aproximação de Funções.

3. Estudos da Implementação e Comparação de Algoritmos de Deep RL:
    - Investigar o algoritmo Deep Q-Networks (DQN) como um baseline, identificando suas limitações teóricas ao lidar com espaços de ações variáveis (restrições de visitação).
    - Implementar métodos de Gradiente de Política (REINFORCE), analisando sua adequação para a otimização direta de permutações estocásticas.
    - Estudar a integração destas abordagens com arquiteturas neurais que respeitam as simetrias topológicas do grafo, especificamente Pointer Networks e Graph Neural Networks (GNNs).

4. Investigação da Estabilidade Teórica (A "Tríade Mortal"): Analisar o impacto da substituição da tabela de valores exata por aproximadores não-lineares. O estudo examinará como a interação entre (1) aproximação de função, (2) bootstrapping (estimativas baseadas em estimativas) e (3) treinamento fora-da-política (off-policy) pode levar à divergência no DQN, fenômeno conhecido na literatura como a "Tríade Mortal", e como o REINFORCE contorna parte destes riscos.

5. Intuição de Convergência via ODE: Utilizar o Método da Equação Diferencial Ordinária (ODE) da teoria de Aproximação Estocástica para fornecer uma interpretação qualitativa da dinâmica de aprendizado. O objetivo é compreender como o Gradiente Descendente Estocástico (SGD) aproxima a trajetória de um sistema dinâmico contínuo em direção a ótimos locais, justificando a escolha de hiperparâmetros como taxas de aprendizado decrescentes.

2. Fundamentação Teórica

Para aplicar o Aprendizado por Reforço (RL) a um problema de otimização combinatória, como o Problema do Caixeiro Viajante (TSP), é necessário estabelecer uma base teórica sólida. Então, serão abordados os conceitos fundamentais do RL ([8]), a começar por sua definição, passando pela sua construção matemática através do Processo de Decisão de Markov (MDP) e sua relação com os Processos de Decisão Determinísticos relacionados ao TSP. Serão definidos os operadores de otimalidade e as propriedades de contração que garantem teoricamente a existência de soluções, diferenciando o cenário de planejamento (modelo conhecido) do cenário de aprendizado. A partir desta modelagem, chega-se a problemas inevitáveis ao escalonar o número de cidades, a "maldição da dimensionalidade", o que justifica a necessidade de abordagens de aprendizado profundo para a realização de aproximação e introduz os desafios de estabilidade que serão analisados posteriormente.

2.1. Aprendizado por Reforço

O Aprendizado por Reforço é um método de aprendizado de máquina, que aborda como um agente computacional pode aprender a tomar decisões por meio da interação direta com um ambiente ([8]). O RL utiliza de recompensas ou punições para que o agente aprenda por meio das interações, diferentemente do aprendizado supervisionado, que se baseia em dados rotulados, também sendo distinto do aprendizado não supervisionado, que busca um modelo de ações baseado em padrões encontrados; o seu objetivo é maximizar a recompensa acumulada ao longo do tempo, por meio da tentativa e erro ([8]).

Contudo é fundamental distinguir entre dois modos de operação dentro deste paradigma: o Planejamento e o Aprendizado. O TSP está localizado, a princípio, no domínio do Planejamento (ou Controle Ótimo Determinístico), uma vez que a dinâmica do ambiente, em métodos tabulares, é plenamente conhecida e determinística. No entanto, devido à intratabilidade computacional de resolver estes modelos exatos em grandes dimensões (onde o espaço de estados cresce fatorialmente), adota-se uma abordagem de Aprendizado, onde a solução é buscada através de amostragem de trajetórias e não da varredura exaustiva de todo o espaço de estados.

Neste contexto, o RL é formalizado como a busca por uma política de controle π que mapeia estados em ações, visando otimizar um retorno. Diferente da otimização clássica que busca uma solução exata e estática, o RL busca uma função de decisão sequencial capaz de generalizar sobre uma variedade de situações dentro deste problema.

A estruturação do problema é baseada nos seguintes elementos principais:

- Agente: A entidade computacional responsável por tomar decisões. No contexto desta pesquisa, o agente não é apenas o executor da rota, mas o aproximador da política que seleciona a permutação de cidades.

- Ambiente: O sistema dinâmico com o qual o agente interage. Para o TSP, seriam o grafo, as cidades, suas coordenadas e as restrições de visitação. O ambiente reage às ações do agente alterando seu estado.

- Estados ($\mathscr{S}_t$): Uma representação compacta da situação atual do ambiente. Para garantir a Propriedade de Markov (Condição essencial para a convergência dos métodos de RL) o estado deve conter todo histórico de informação necessária para tomada de decisão futura. No TSP, isso implica em codificar não apenas o estado atual mas o conjunto de cidades já visitadas.

- Ações ($\mathscr{A}_t$): As escolhas disponíveis para o agente em um determinado estado. O espaço de ações no TSP é dinâmico e depende do estado, restrito ao conjunto de cidades não visitadas.

- Dinâmica ou Modelo de Transição: A regra que governa a evolução do sistema. Embora o TSP seja fundamentalmente um Processo de Decisão Determinístico (onde a ação de ir para uma cidade resulta nessa transição com probabilidade 1), o RL trata a exploração desse espaço de forma estocástica. A dinâmica define como o estado $\mathscr{S}_t$ e a ação $\mathscr{A}_t$ produzem o estado seguinte $\mathscr{S}_{t+1}$.

- Recompensa ($\mathcal{R}_t$): Um sinal escalar imediato recebido após cada transição, servindo como um gradiente de desempenho local. No TSP define-se normalmente como o negativo da distância euclidiana entre as cidades, fazendo a maximização do retorno acumulado ser equivalente a minimização do custo total da rota ([5, 6, 9]).

- Horizonte (T): O tempo ou número de passos até o término do processo. O TSP é modelado como uma tarefa episódica de horizonte finito, onde T é igual ao número de cidades N, encerrando-se quando o ciclo pelas cidades é completo.

2.2. Processos de Decisão de Markov (MDP) e SSP

A formalização matemática padrão para a modelagem de tomada de decisão sequencial sob incerteza é o Processo de Decisão de Markov (MDP). Contudo, dada a natureza do TSP que possui um estado terminal objetivo (todas as cidades visitadas) o problema se enquadra mais precisamente na definição de um Problema de Caminho Mias Curto Estocástico (*Stochastic Shortest Path* - SSP) ou um MDP Episódico.

Um MDP (e por extensão, um SSP) é formalmente definido pela tupla ($\mathscr{S}$, $\mathscr{A}$, $\mathscr{P}$, r, γ), onde ([8]):

- $\mathscr{S}$ é o espaço de todos os estados possíveis, que no TSP cresce fatorialmente com o número de cidades.

- $\mathscr{A}$ é o espaço de ações possíveis, onde $\mathscr{A}$(s) denota o conjunto de ações possíveis no estado s.

- $\mathscr{P} : \mathscr{S} \times \mathscr{A} \rightarrow \Delta(\mathscr{S})$ é a função de probabilidade de transição, onde $p(s' | s, a)$ define a probabilidade de transitar para o estado s' ao executar a ação a no estado s. Em ambientes determinísticos a distribuição de probabilidade se torna 1.

- r : $\mathscr{S} \times \mathscr{A} \rightarrow \mathbb{R}$ é a função de recompensa esperada que está integrada na dinâmica do ambiente. A recompensa r(s,a) é o retorno numérico imediato ao realizar certa ação a em um estado s.

- $\gamma \in [0, 1]$ é o fator de desconto. Embora problemas de horizonte finito como o TSP permitam $\gamma = 1$ (sem desconto), a utilização de $\gamma < 1$ é frequentemente utilizada em métodos de aproximação para assegurar propriedades teóricas de contração do operador de Bellman.

A premissa central de um MDP é a Propriedade de Markov, que estabelece que o futuro é condicionalmente independente do passado, dado o presente. Matematicamente, isso implica que a dinâmica do sistema depende apenas do estado atual s_t e da ação a_t, não da trajetória anterior de estados e ações. O objetivo do agente é aprender uma política $\pi : \mathcal{S} \rightarrow \Delta(\mathcal{A})$ (um mapeamento de estados para probabilidade de seleção de ações) que maximize o retorno esperado acumulado.

Para avaliar a eficácia de uma política π, define-se duas funções fundamentais, conhecidas como funções de valor ([6, 8]):

- A **Função de Valor de Estado** (V^{π}(s)) representa o retorno esperado ao iniciar no estado s e seguir a política π para todas as decisões subsequentes:

  $$V^{\pi}(s) = \mathbb{E}_{\pi}\left[\sum_{k=0}^{\infty} \gamma^{k} \boldsymbol{\mathcal{R}}_{t+k+1} \mid \mathcal{S}_{t} = s\right]$$

  onde $\mathbb{E}_{\pi}$ é o valor esperado ao seguir a política π.

- A **Função de Valor de Ação** (Q^{π}(s, a)) representa o retorno esperado ao iniciar no estado s, tomar a ação e, em seguida, seguir a política π:

  $$Q^{\pi}(s, a) = \mathbb{E}_{\pi}\left[\sum_{k=0}^{\infty} \gamma^{k} \boldsymbol{\mathcal{R}}_{t+k+1} \mid \mathcal{S}_{t} = s, \mathcal{A}_{t} = a\right]$$

A relação fundamental que permite o cálculo destas funções não é através da soma infinita direta, mas sim com a sua decomposição recursiva, conhecida como Equação de Bellman. Para a função de valor de estado, a equação decompõe o valor em duas partes, a recompensa imediata esperada e o valor descontado do próximo estado:

$$V^{\pi}(s) = \sum_{a \in \mathcal{A}} \pi(a|s) \sum_{s' \in \mathcal{S}} \mathcal{P}(s'|s, a)\,[r(s, a) + \gamma V^{\pi}(s')]$$

Para fins de análise de convergência e estabilidade (mais discutido no capítulo 5), é apropriado expressar esta relação recursiva através de operadores funcionais. Defini-se o Operador de Bellman para uma política fixa, denotado por T^{π}, que mapeia uma função de valor V em outra função de valor $T^{\pi}V$ sobre o mesmo espaço de estados $\mathscr{S}$. A aplicação deste operador é definida como:

$$(T^{\pi}V)(s) = \mathbb{E}_{a \sim \pi(\cdot|s),\, s' \sim \mathcal{P}(\cdot|s,\, a)}\,[r(s, a) + \gamma V(s')]$$

nesta formulação, o operador de esperança $\mathbb{E}$ adiciona a incerteza pertencente ao processo de decisão sobre duas variáveis aleatórias. O termo a ~ π(• | s) indica que a ação é amostrada estocasticamente a partir da política do agente, um mecanismo fundamental durante o treinamento para garantir a exploração do espaço de soluções. Da mesma forma, o termo s' ~ $\mathscr{P}$(• | s, a) representa a incerteza na dinâmica do ambiente. Embora no TSP clásico as transições sejam determinísticas, esta notação probabilística generaliza o modelo para variantes estocásticas e preserva o rigor da formulação de MSPs, onde o valor de um estado é definido pela média ponderada de todos os retornos futuros possíveis.

Desta forma a função de valor verdadeira V^{π} é caracterizada como um ponto fixo único deste operador, satisfazendo a equação funcional $T^{\pi}V^{\pi} = V^{\pi}$. A existência e unicidade desta solução são garantidas pelo fato de T^{π} ser um operador de contração a norma máxima ($\| \cdot \|_{\infty}$) para $\gamma < 1$, uma propriedade que sustenta os algorítmos iterativos de avaliação de política.

2.2.1. Políticas e Funções de Valor Ótimas

Resolver um problema de RL significa encontrar uma política que maximize o retorno esperado a longo prazo, de modo que uma política π é considerada melhor ou igual a outro política π' se $V^{\pi}(s) \geq V^{\pi'}(s)$ para todos os estados $s \in \mathscr{S}$. Existe sempre pelo menos uma política que é melhor ou igual a todas a outras, esta é chamada de política ótima, denotada por π^* ([8]).

As políticas ótimas compartilham as mesmas funções de valor, denominadas Função de Valor de Estado Ótima (V^*) e Função de Valor de Ação Ótima (Q^*), definidas como ([8]):

- $V^*(s) = \max_{\pi} V^{\pi}(s)$
- $Q^*(s, a) = \max_{\pi} Q^{\pi}(s, a)$

Estas funções satisfazem as Equações de Otimalidade de Bellman ([8]). Diferentemente das equações de avaliação para uma política fixa (que são lineares), as equações de otimalidade introduzem uma não-linearidade através do operadoe de maximização. Desta forma, o Operador de Otimização de Bellman (T^*) define-se, em termos de operadores funcionais, como:

$$(T^{*}V)(s) = max_{a \in \mathcal{A}} \left\{ r(s, a) + \gamma \sum_{s' \in \mathcal{S}} \mathcal{P}(s' \mid s, a)\, V(s') \right\}$$

A função de valor ótima V^* é o único ponto fixo deste operador, isto é, $T^{*}V^{*} = V^{*}$. A existência e unicidade desta solução são garantidas pelo Teorema de Ponto Fixo de Banach, visto que T^* atua como uma γ-contração na norma infinito (para $\gamma < 1$). Isso fornece a base teórica para algorítmos como *Value Iteration* e *Q-Learning*, que buscam este ponto fixo iterativamente.

A relação entre V^* e Q^* permite expressar a otimalidade em termos da seleção da melhor ação:

$$Q^{*}(s, a) = \mathbb{E}\left[\boldsymbol{\mathcal{R}}_{t+1} + \gamma V^{*}(\mathcal{S}_{t+1}) \mid \mathcal{S}_{t} = s, \mathcal{A}_{t} = a \right]$$

Uma vez obtida a função Q^*(s, a) (ou V^*), a política ótima π^* pode ser derivada diretamente agindo de forma gulosa (*greedy*) em relação aos valores ótimos. No caso determinístico ou com Q^*, a política ótima é determinística:

$$\pi^*(s) = arg\,max_{a \in \mathcal{A}} Q^*(s, a)$$

Esta equação afirma que, sob a otimalidade, a melhor ação é aquela que maximiza a soma da recompensa imediata com o valor descontado do melhor estado futuro possível. Encontrar uma solução para estas equações (aproximando Q^* sem conhecer $\mathscr{P}$) é o objetivo dos algoritmos de RL baseados em valor como o DQN.

2.3. Modelando o TSP como um MDP

A aplicação de Aprendizado por Reforço ao TSP exige a tradução de um problema de otimização combinatória estática para a estrutura sequencial de um MDP. Dada a existência de um objeto terminal definido (visitar todas as cidades e retornar à origem), o problema é mais rigorosamente classificado como um Problema de Caminho Mais Curto Estocástico (*Stochastic Shortest Path* - SSP). Nesta formulação, busca-se uma política que minimize o custo acumulado até alcançar um estado alvo.

Os componentes do SSP para o TSP são definidos da seguinte maneira:

- Estados ($\mathscr{S}$): Para satisfazer a Propriedade de Markov ([8]), o estado $\mathscr{S}_t$ deve encapsular toda a informação necessária para a tomada de decisão futura sem depender do histórico passado. No TSP, define-se o estado como uma tupla $\mathscr{S}_t = (x_t, \mathscr{V}_t)$, onde x_t representa a cidade atual do agente e $\mathscr{V}_t$ é o conjunto (ou vetor binário) das cidades já visitadas, definição necessária para satisfazer a propriedade de Markov. O estado terminal $\mathscr{S}_G$ é alcançado quando $|\mathscr{V}_t| = N$ e o agente retorna à cidade de origem. O tamanho deste espaço de estados cresce a taxa de ($N * 2^N$), justificando o uso de aproximações de funções ([5, 8]).

- Ações ($\mathscr{A}$(s)): O conjunto de ações admissível depende do estado atual. Em um estado $s_t = (x_t, \mathscr{V}_t)$, a ação a_t consiste em escolher a próxima cidade a visitar. Para garantir a validade da rotam o espaço de ações é restrito dinamicamente: $\mathcal{A}(s_t) = \{c \mid c \notin \mathcal{V}_t\}$, ou seja, apenas cidades não visitadas são elegíveis ([5, 9]).

- Dinâmica de Transição ($\mathscr{P}$): A dinâmica do ambiente do TSP é determinística. Dado um estado s_t e uma ação válida a_t (ir para a cidade j), o próximo estado s_{t+1} é determinado com probabilidade 1: a cidade atual torna-se j e o conjunto $\mathscr{V}_{t+1}$ é atualizado para $\mathscr{V}_t \cup \{j\}$. Contudo, algoritmos de RL frequentemente induzem estocasticidade na seleção de ações para garantir a exploração adequada do espaço de estados durante o treinamento.

- Função de Recompensa e Custo (r): No formalismo do SSP, o objetivo é minimizar o custo acumulado. Define-se o custo imediato de uma transição como a distância euclidiana (ou ponderada) entre as cidades: $c(s, a) = d(x_t, x_{t+1})$. Para compatibilidade com algoritmos de maximização de retorno, utiliza-se a recompensa como o negativo do custo: $r(s, a) = -d(x_t, x_{t+1})$. Isso garante que a maximização do retorno esperado (solução do MDP) sema matematicamente equivalente à minimização do comprimento da rota ([5]).

- Fator de Desconto (γ) e Estabilidade: Teoricamente, problemas de horizonte finito ou SSPs permitem o uso de $\gamma = 1$ (sem desconto), desde que se garanta que a política é "própria" (alcança o estado terminal com probabilidade 1). No entanto, na prática de aproximação com redes neurais, é comum utilizar um fator de desconto $\gamma \in [0, 1)$ (por exemplo, 0.99). Isso introduz um viés heurístico que favorece recompensas mais imediatas, mas é frequentemente necessário para assegurar a propriedade de contração do Operador de Bellman aproximado, facilitando a convergência numérica dos algoritmos de aprendizado ([8]).

Com esta formulação, a Equação de Otimalidade de Bellman para o TSP pode ser descrita de forma explícita. Dado que o estado $s_t = (c_i, \mathscr{V}_t)$, onde c_i é a cidade atual e $\mathscr{V}_t$ é o conjunto de cidades visitadas, ao executar uma ação a_t que leva à cidade c_j, o valor Q^* deve satisfazer a condição de consistência recursiva. Diferente de uma simples soma de custos, a equação de otimalidade exige que o valor atual considere a melhor decisão possível a partir do próximo estado. Então a equação se torna:

$$Q^*((c_i, \mathscr{V}_t), c_j) = -d(c_i, c_j) + \gamma \max_{c_k \notin \mathcal{V}t \cup \{c_j\}} Q^*((c_j, \mathscr{V}_t \cup \{c_j\}), c_k)$$

Esta equação ilustra a subestrutura ótima do problema: a solução ótima para o TSP contém dentro de si soluções ótimas para subproblemas de rotas menores. Esta propriedade estabelece uma conexão profunda com algoritmos clássicos de Programação

Dinâmica, especificamente o algoritmo de Held-Karp. Este algoritmo resolve o TSP de forma exata ao computar sistematicamente os valores Q^* (ou V^*) para subconjuntos crescentes de cidades, garantindo a descoberta do ponto fixo do Operador de Bellman.

Embora o algoritmo de Held-Karp reduza a complexidade fatorial da força bruta (O(N!)) para uma complexidade exponencial de ordem $O(N^2\ 2^N)$, ele ainda se torna inviável para problemas de escala realista. Isso ocorre porque o vetor de valores V^*, necessários para armazenar a solução exata, possui uma dimensão igual à cardinalidade do espaço de estados $|\mathcal{S}|$. Como $|\mathcal{S}|$ cresce exponencialmente, o armazenamento e a varredura necessários para aplicar o Operador de Bellman exato tornam-se computacionalmente intratáveis, fenômeno que motiva a transição de métodos exatos para métodos de Aprendizado por Reforço com Aproximação de Funções.

3. A Necessidade de Aproximação

3.1. A Maldição da Dimensionalidade

Embora o TSP possa ser formalmente modelado como um MDP finito, uma barreira prática surge, a “maldição da dimensionalidade” ([8]). Este termo, usado por Record Bellman, refere-se ao crescimento de estados $|\mathcal{S}|$ do TSP, de modo que sendo definido pela cidade atual e pelo conjunto de cidades visitadas, para um problema com N cidades:

- O número de escolhas para a cidade atual é, no máximo, N.
- O número de subconjuntos de cidades visitadas é 2^N.

Portanto, o número total de estados é da ordem de $O(N\ 2^N)$, desta forma, para um problema com N = 20 cidades, o número de estados é superior a 20 milhões e para N = 50, o número torna-se computacionalmente intratável. Esta explosão combinatória tem duas consequências diretas que impedem a solução exata do MDP através dos métodos tabulares apresentados nas seções anteriores:

1. Inviabilidade de Memória: Ao considerar métodos tabulares, como o Q-learning, que exigem o armazenamento de uma tabela de valores (a Q-table) variando para cada par estado-ação, com o tamanho máximo desta matriz chegando a dimensão $|\mathcal{S}| \times |\mathcal{A}|$, excedendo a capacidade até dos computadores modernos para N grande.

2. Inviabilidade Computacional: A aplicação do Operador de Bellman exato T^*exige varreduras completas sobre todo o espaço de estados para computar esperanças matemáticas. Então o custo computacional de cada varredura é, no mínimo, proporcional a $|\mathcal{S}|$ (da ordem de $O(N\ 2^N)$), sendo inviável ([8]).

Para superar essa intratabilidade, substitui-se a representação tabular exata $V(s)$ por uma Aproximação de Função Parametrizada $V(s, \omega)$, onde $\omega \in \mathbb{R}^d$ é um vetor de pesos de dimensão muito inferior à do espaço de estados ($d \ll |\mathcal{S}|$). No contexto deste trabalho essa aproximação é realizada por Redes Neurais Profundas (*Deep RL*).

3.2. O Operador de Projeção

A introdução da aproximação altera as propriedades teóricas fundamentais do problema. O objetivo deixa de ser encontrar o ponto fixo exato das equações de Bellman (V^π) e passa a ser encontrar a melhor aproximação possível dentro do espaço parametrizado pela rede neural. Geometricamente, cada atualização do algoritmo pode ser vista como a aplicação do Operador de Bellman seguida por uma projeção Π da função resultante de volta ao subespaço representável pelos parâmetros w, então a função $V(s, \omega)$ restringe a solução a um subespaço de baixa dimensão dentro do espaço original de todas as funções de valor possíveis. Essa operação composta ΠT é a raiz das instabilidades teóricas discutidas posteriormente, pois, o objetivo do algoritmo deixa de ser encontrar o ponto fixo V^* (que pode não residir no subespaço aproximado) e passa a ser encontrar a melhor aproximação possível dentro da família de funções representáveis pela rede neural.

Matematicamente isso introduz um Operador de Projeção Π, que mapeia uma função de valor qualquer para o ponto mais próximo no espaço de parâmetros. A iteração de aprendizado passa a tentar resolver a Equação de Bellman Projetada (PBE):

$$\Phi\omega = \Pi T(\Phi\omega)$$

A distinção, que será explorada na análise de convergência (Capítulo 5), é que enquanto o operador original T é uma contração na norma máxima ($\mathbb{L}_\infty$), o operador composto ΠT não oferece necessariamente garantias de contração, especialmente quando combinado com *bootstrapping* e amostragem *off-policy*. Essa perda de garantias teórica é a fonte primária de instabilidade em métodos de *Deep* RL e motiva o estudo de mecanismos de estabilização como *Replay Buffers* e *Target Networks*.

3.3. Viés Indutivo e Representação de Grafos

3.3.1. A Natureza Não-Euclidiana dos Dados

A eficácia da generalização em modelos de aprendizado profundo depende do alinhamento entre a arquitetura da rede neural e a estrutura dos dados, um conceito chamado Viés Indutivo (*Inductive Bias*). Enquanto arquiteturas convolucionais (CNNs) obtiveram progresso marcante em visão computacional ao explorar a estrutura de grade regular (Euclidiana), problemas de Otimização Combinatória como o TSP operam sobre dados com estrutura de grafo irregular e não-Euclidiana. Neste domínio, a informação relevante não reside na posição absoluta de um nó em um vetor de entrada, mas sim nas relações topológicas e nas distâncias relativas entre os vértices conectados.

A utilização de redes neurais tradicionais (feedforward, MLPs) ou Redes Neurais Recorrentes (RRNs) puras impõe limitações severas à representação de estados em problemas de grafos. Uma instância do TSP com N cidades pode ser representada por N! permutações diferentes de sua matriz de adjacência ou lista de coordenadas; contudo, todas essas representações descrevem um grafo isomorfo com a mesma solução ótima. Modelos tradicionais não possuem o viés indutivo geométrico necessário para reconhecer essa equivalência automaticamente, o que resulta em uma ineficiência amostral e falha na generalização para instâncias de tamanhos variados.

3.3.2. Formalização de Simetrias

Para mitigar a explosão combinatorial do espaço de estados, é essencial restringir o espaço de hipóteses da rede neural impondo propriedades geométricas de simetria, especificamente a invariância e a equivariância à permutação. A invariância à permutação é exigida na aproximação da função de valor (V(s)), garantindo que a estimativa do custo restante da rota permaneça inalterada independentemente da ordem de indexação das cidades, ou seja, f(PX) = f(X), onde P é uma matriz de permutação. Já a equivariância à permutação é fundamental para a política (π(a|s)), pois assegura que uma reordenação na entrada (como a disposição dos nós) resulte em uma reordenação correspondente na saída (a distribuição de probabilidades sobre cidades candidatas), formalmente expressa como (f(PX) = Pf(X)). A ausência dessa propriedade em arquiteturas tradicionais força o modelo a memorizar mapeamentos específicos de índices, impedindo a generalização e a transferência de aprendizado entre grafos topologicamente idênticos, mas com numeração de nós distinta, o que compromete a eficiência e robustez da solução.

3.3.3. Graph Neural Networks (GNNs)

As Redes Neurais de Grafos (GNNs) são uma classe de arquiteturas projetadas para operar nativamente em dados estruturados como grafos, incorporando propriedades geométricas como invariância e equivariância à permutação, essenciais para problemas de otimização combinatória como o do Caixeiro Viajante (TSP). Elas funcionam por meio de um mecanismo iterativo de passagem de mensagens, no qual a representação vetorial (embedding) de cada nó é atualizada com base nas informações de seus vizinhos, utilizando operações de agregação comutativas (como soma ou média que garantem a invariância à ordem dos nós. Esse processo permite que as GNNs capturem a estrutura topológica do grafo independentemente de sua representação matricial. No contexto de aprendizado por reforço, as GNNs projetam o grafo de entrada em um espaço latente rico, gerando embeddings que codificam características locais e globais do problema, os quais servem como base para mecanismos de atenção que definem a política do agente, facilitando a descoberta de heurísticas generalizáveis para tarefas de roteamento e otimização.

3.4. Arquiteturas Especializadas: Pointer Networks e Attention

3.4.1. Superando o Vocabulário Fixo

As arquiteturas tradicionais de aprendizado profundo operam sobre um vocabulário de saída fixo e predefinido. No entanto, o TSP apresenta um desafio singular: o 'vocabulário' de saída (as cidades a serem visitadas) varia a cada instância e seu tamanho é igual ao tamanho da entrada N.

Para superar esta limitação foram introduzidas as Redes de Ponteiros, diferentemente de classificadores que mapeiam estados a índices fixos, esta arquitetura utiliza um mecanismo de atenção para gerar uma distribuição de probabilidade sobre os próprios vetores de entrada. O modelo opera como um decodificador que, a cada passo, 'aponta' para um dos nós do grafo de entrada como a próxima cidade da rota, permitindo que a rede generalize para instâncias com número de cidades N diferente daquele observado durante o treinamento.

3.4.2. Mecanismo de Atenção

Enquanto *Pointer Networks* originais processava as cidades uma por uma em sequência, pesquisas mais recentes adotam o Mecanismo da Atenção (baseado em *Transformers*). A intuição aqui é similar à visão humana: ao olhar para um mapa, não focamos em todas as cidades com a mesma intensidade, já filtramos por proximidade.
O modelo funciona em duas etapas:

1. Codificador (*Encoder*): A rede analisa o grafo inteiro de uma vez e cria uma "assinatura" (*embedding*) para cada cidade. Essa assinatura contém não apenas a posição da cidade, mas também quem são seus vizinhos mais próximos.
2. Decodificador (*Decoder*): Na hora de construir a rota, a rede calcula um "*score* de atenção" entre a cidade atual (onde o agente está) e todas as outras cidades não visitadas. Essa pontuação representa a afinidade ou atração entre elas. Se a cidade A está muito perto da cidade B, o *score* de atenção será alto, indicando uma forte probabilidade de B ser o próximo passo.

3.4.3. Garantia de Validez via Mascaramento (Masking)

Uma regra fundamental no TSP é que cada cidade deve ser visitada exatamente uma vez (Ciclo Hamiltoniano). O algoritmo DQN padrão frequentemente falha em respeitar esta restrição sem penalidades complexas na função de recompensa. Nas arquiteturas modernos de Otimização Combinatória Neural, essa restrição é imposta através do mecanismo de mascaramento (*masking*). Antes da aplicação da função *Softmax* (que transforma as pontuações em distribuições de probabilidade, através de exponenciais dos valores) os valores correspondentes às cidades já visitadas são forçados ao $-\infty$, o que garante que a probabilidade de selecionar tais cidades seja igual a zero ($\exp(-\infty) \approx 0$), assegurando que o agente amostre apenas transições válidas e evitando a necessidade de aprender rotas possíveis por tentativa e erro.

4. Metodologia de Algoritmos de Aprendizado por Reforço Profundo

A formulação do Problema do Caixeiro Viajante (TSP) como um Processo de Decisão Sequencial, embora teoricamente elegante, na prática é inviável devido ao desafio imposto pela “maldição da dimensionalidade”. Conforme discutido anteriormente, o

crescimento do espaço de estados, que é da ordem de $O(N^2\ 2^N)$, inviabiliza métodos tabulares exatos de Programação Dinâmica. Nestes cenários o custo computacional para varrer o espaço de estados e a e exigência de memória para armazenar a tabela de valores (Q-table) tornam-se intratáveis para instâncias de tamanho realista.

A solução metodológica para este desafio é a aproximação de funções (*function approximation*). Em vez de armazenar um valor para cada estado, utilizamos uma função parametrizada, como uma Rede Neural Profunda (DNN), para estimar a função de valor; por exemplo, $V(s) \approx V^\wedge(s, \omega)$ ou, para representar a política, $\pi(a|s) \approx \pi(a|s, \theta)$. O vetor de parâmetros (ω ou θ) possui uma dimensão significativamente menor do que a cardinalidade do espaço de estados ($|\omega| << |\mathcal{S}|$), permitindo a generalização do aprendizado para estados não visitados([8]).

Entretanto, essa transição altera fundamentalmente a natureza do problema de otimização. Ao introduzir um aproximador parametrizado, restringimos a busca pela solução ótima a um subespaço de funções representáveis pela arquitetura da rede neural escolhida. Matematicamente, isso implica que a aplicação dos operadores de Bellman (definidos no Capítulo 3) passa a ser seguida por uma operação de projeção ortogonal ou aproximada sobre este subespaço. O objetivo do algoritmo deixa de ser encontrar o ponto fixo exato das equações de Bellman e passa a ser a minimização de uma função de erro ou a maximização direta do desempenho esperado dentro das limitações da arquitetura.

Esta abordagem, denominada Aprendizado por Reforço Profundo (*Deep Reinforcement Learning*, DRL), organiza-se em duas famílias metodológicas principais que lidam com essa aproximação de maneiras distintas:

1. Métodos Baseados em Valor (*Value-based Methods*): Estes algoritmos, exemplificados pelo *Deep Q-Networks* (DQN), concentram-se em aprender uma função de valor de ação $Q^*(s, a)$. A política é derivada implicitamente desta função, tipicamente selecionando a ação que maximiza o valor estimado (*greedy policy*). O desafio central aqui é minimizar o erro de predição temporal (Erro TD) garantindo que a projeção da função de valor não induza divergência.

2. Métodos Baseados em Política (*Policy-based Methods*): Estes algoritmos visam aprender e otimizar a política $\pi(a|s, \theta)$ diretamente, sem necessariamente passar por uma função de valor intermediária para a seleção de ações. O objetivo é ajustar os parâmetros θ para maximizar diretamente o retorno esperado, $J(\theta)$ através de subida de gradiente. Algoritmos como o REINFORCE e estruturas Ator-Crítico (*Actor-Critic*) pertencem a esta categoria.

Ambas famílias seguem o princípio da Iteração de Política Generalizada (*Generalized Policy Iteration*, GPI), onde os processos de avaliação (estimar o quão boa é a política atual) e melhoria (tornar a política mais gulosa em relação a estimativas) interagem continuamente. A seguir, detalham-se os algoritmos específicos e as arquiteturas neurais especializadas para dados estruturados em grafos (GNNs) que compõem a metodologia desta pesquisa.

4.1. Métodos Baseados em Valor: *Deep Q-Networks* (DQN)

Os métodos de Aprendizado por Reforço Profundo (DRL) Baseados em Valor fundamentam-se na premissa de que uma política ótima pode ser derivada implicitamente através da estimativa precisa da Função de Valor de Ação ótima (Q^*). O algoritmo *Deep Q-Network* (DQN) representa a evolução canônica desta classe, estendendo o algorítmo tabular *Q-learning* ao substituir a tabela de valores exata por um aproximador de função não linear parametrizado, tipicamente uma Rede Neural Profunda.

Neste contexto, o DQN utiliza uma rede neural (denominada *Q-network*) para estimar a função de valor ótima, $Q^*(s, a) \approx Q\hat{}(s, a, \omega)$, onde ω representa o vetor de pesos treináveis ([8]). O objetivo teórico do algoritmo é ajustar iterativamente ω para que a função aproximada convirja para o ponto fixo do Operador de Otimalidade de Bellman (T^*), definido formalmente no Capítulo 3. Diferentemente de métodos de regressão supervisionada onde o alvo é fixo, no DQN o alvo é móvel e derivado das próprias estimativas da rede, um processo conhecido como *bootstrapping*.

O DQN é classificado como um algoritmo livre de modelo (*model-free*), pois aprende a utilidade das ações diretamente das transições amostradas, sem a necessidade de estimar explicitamente as probabilidades de transição $\mathscr{P}$ do ambiente. Além disso, opera em regime fora-da-política (*off-policy*), o que permite ao agente aprender a função de valor ótima Q^* enquanto segue uma política de comportamento exploratória (como ϵ-*greedy*), utilizando o Mascaramento de Ações para evitar a exploração de cidades já visitadas. Esta característica é particularmente vantajosa para o TSP, pois permite que o agente reutilize experiências passadas armazenadas em um *buffer* de repetição, melhorando a eficiência de amosta e desacoplando as correlações temporais das trajetórias.

A aplicação do DQN ao TSP impõe um desafio arquitetural específico: a representação do estado. Como o estado do TSP é definido por um grafo, a função de valor deve respeitar propriedades de simetria geométrica; especificamente, o valor de um estado

não deve depender da ordem em que os nós (cidades) são listados na entrada, mas sim de suas relações topológicas e distâncias. Para satisfazer essa condição de invariância a permutação, utiliza-se a estratégia de *Graph Embedding* (Incorporação de Grafos).

Em vez de processar coordenadas brutas, o DQN para Otimização Combinatória (CO) emprega Redes Neurais de Grafos (GNNs), como a arquitetura *Structure2Vec* (S2V) ou Graph Attention Networks (GAN), para parametrizar a função Q. A GNN mapeia a estrutura do grafo e o estado parcial da solução (cidades visitadas) em um vetor de características (*embeddings*) de baixa dimensão $\phi(s)$. ([5])

A função de valor é então aproximada sobre essas características:

$$Q(s,\ a;\ \omega)\ \approx\ f(\phi(s),\ a;\ \omega)$$

Essa abordagem de "aprendizado principal" (*principal learning*) permite que o agente construa a solução passo a passo, recebendo recompensas (negativas das distâncias) e aprendendo a generalizar a política de construção para grafos de tamanhos e topologias variadas, superando as limitações de entrada fixa das redes neurais tradicionais.

4.1.1. O Mecanismo de Aprendizado por Diferença Temporal

A essência do DQN está na utilização do erro de Diferença Temporal (*Temporal Difference*, TD) como um sinal de aprendizado para guiar a otimização dos pesos da rede neural. Diferentemente dos métodos de Monte Carlo, que exigem o término de um episódio para calcular o retorno real G_t, o método de TD aprende a partir de transições incompletas, uma técnica conhecida como *bootstrapping*.

O algoritmo DQN busca fazer com que a função aproximada $Q(s, a; \omega)$ convirja para a função ótima $Q^*(s, a; \omega)$, que é o ponto fixo das Equações de Otimalidade de Bellman (definidas na Seção 3.2.1). Para isso, a cada passo de tempo t, o algoritmo calcula um Alvo TD (Y_t), que serve como um substituto ("*proxy*") para o valor verdadeiro desconhecido.

Matematicamente, o cálculo do alvo Y_t pode ser interpretado como uma aplicação amostral do Operador de Otimalidade de Bellman (T^*) sobre a estimativa de valor atual. O alvo é definido como a recompensa imediata mais o valor descontado do melhor estado futuro estimado:

$$Y_t = r(s_t, a_t) + \gamma \max_{a' \in \mathcal{A}} Q^{\wedge}(s_{t+1}, a'; \omega^-)$$

Em arquiteturas de *Deep* RL estáveis, o termo de bootstrapping $\max_{a' \in \mathcal{A}} Q^{\wedge}(s_{t+1}, a'; \omega^{-})$ é frequentemente calculado utilizando um conjunto de pesos ω^{-}, as atualizações poderiam criar ciclos de retroalimentação instáveis, onde o aumento do valor $Q(s, a)$ aumentaria simultaneamente o alvo, levando à divergência ou oscilação dos valores estimados.

O Erro TD (δ_t) quantifica a discrepância temporal entre a predição atual da rede para o estado s_t e a nova estimativa refinada Y_t obtida após observar a transição:

$$\delta_t = Y_t - Q^{\wedge}(s, a; \omega)$$

Este erro δ_t representa uma amostra do Erro de Bellman, indicando o quão longe a função de valor atual está de satisfazer a condição de consistência de Bellman. O processo de aprendizado consiste, portanto, em minimizar a expectativa quadrática deste erro ao longo das trajetórias experimentadas pelo agente.

4.1.2. Otimização do Gradiente Estocástico e o Erro TD

O processo de aprendizado do DQN é formalizado como a minimização da discrepância entre a função de valor aproximada $Q(s, a; \omega)$ e o alvo de Bellman estimado. Define-se a função de perda $L_t(\omega)$ para uma transição amostrada no instante t como o Erro Quadrático Médio (*Mean Squared Error*, MSE) do erro de Diferença Temporal:

$$L_t(\omega) = L_t(\omega) = \frac{1}{2}\left[Y_t - Q(s_t, a_t; \omega)\right]^2$$

O ajuste dos parâmetros ω é realizado utilizando algoritmos de otimização baseados na Descida de Gradiente Estocástico (*Stochastic Gradient Descent*, SGD). O objetivo é mover os pesos na direção oposta ao gradiente da função de perda para minimizar o erro local. A regra de atualização, onde α é a taxa de aprendizado (*learning rate*), é dada por:

$$\omega_{t+1} = \omega_t - \alpha \nabla_{\omega} L_t(\omega)$$

Ao aplicar a regra da cadeia para calcular o gradiente $\nabla_{\omega} L_t(\omega)$, surge uma distinção teórica fundamental. Embora o alvo Y_t dependa dos pesos da rede (diretamente ou via de rede alvo), o algoritmo DQN trata Y_t como uma constante fixa durante o passo de diferenciação. Isso caracteriza o método não como uma descida de gradiente pura, mas como uma Descida de Semi-Gradiente. A regra de atualização resultante expressa como:

$$\omega_{t+1} = \omega_t + \alpha\left[Y_t - Q(s_t, a_t; \omega_t)\right]\nabla_\omega Q(s_t, a_t; \omega_t)$$

tal que ∇ q^(s_t, a_t, ω) é o vetor gradiente do valor estimado em relação aos pesos ω.

Nesta equação, o termo [Y_t - Q(s_t, a_t; ω_t)] representa o erro escalar δ_t, enquanto $\nabla_\omega Q$(s_t, a_t; ω_t) indica a sensibilidade da função de valor em relação aos parâmetros. Essa abordagem de semi-gradiente simplifica o cálculo computacional, mas conforme será discutido na análise de convergência (Capitulo 5) isso remove as garantias de convergência monótona para o ótimo global que existiriam em problemas de regressão supervisionada estacionária, introduzindo riscos de oscilação e divergência em regimes de aproximação não-linear ([5, 6]).

4.1.3. Mecanismos de Estabilidade Adicionais

A aplicação direta da atualização de gradiente apresentada acima em redes neurais profundas enfrenta desafios severos de instabilidade. Teoricamente, isso ocorre porque o algoritmo DQN viola duas premissas fundamentais da otimização estocástica clássica: as amostras de treinamento não são independentes e identicamente distribuídas (i.i.d.), e o alvo de aprendizado é não estacionário. Para mitigar esses riscos e aproximar o comportamento do algoritmo às condições de convergência teórica, empregam-se dois mecanismos de estabilização essenciais:

1. Redes Alvo (*Target Networks*): No Q-*Learning* padrão, a atualização dos pesos ω altera imediatamente tanto a estimativa de valor Q(s, a; ω) quanto o alvo Y_t usado para calcular o erro. Isso cria um problema de “alvo móvel” (*moving target*), onde o algoritmo tenta perseguir uma função objetivo que muda a cada iteração, podendo levar a oscilação e divergência dos parâmetros. Para estabilizar este processo o DQN utiliza uma Rede Alvo (*target network*) separada, com parâmetros ω^-, para calcular o valor dos estados futuros (Assim como apresentado no Capítulo 4.1.1).
   Os pesos ω^- são mantidos congelados por um número fixo de passos C e, periodicamente, são sincronizados com os pesos da rede principal ($\omega^- \leftarrow \omega$). Matematicamente isso converte o problema de Aprendizado por Reforço instável em uma sequência de problemas de Regressão Supervisionada estacionários, onde o alvo permanece fixo durante intervalos curtos, facilitando a convergência do otimizador SGD.

2. Repetição de Experiência (*Experience Replay*): Em um processo de decisão sequencial, as amostras de transição consecutivas (s_t, a_t, r_t, s_{t+1}) são altamente correlacionadas. O treinamento direto sobre essas sequências viola a premissa de dados i.i.d. exigida pelos métodos de gradiente estocástico, enviesando a rede para distribuição de estados mais recentes e causando o "esquecimento" de experiências passadas.Para contornar isso o agente armazena as transições em um Buffer de Repetição $\mathscr{D}$ com capacidade finita. A cada passo de treinamento um lote de transições é amostrado aleatoriamente de $\mathscr{D}$ para calcular a perda e atualizar os pesos:

$$L_t(\omega) = \mathbb{E}_{(s,a,r,s')\sim U(\mathcal{D})}\left[\left(r + \gamma \max_{a'\in\mathcal{A}} Q\left(s', a'; \omega^-\right) - Q(s, a; \omega)\right)^2\right]$$

Esta técnica cumpre os dois papéis teóricos cruciais: quebra a correlação temporal entre amostras sucessivas, reduzindo a variância da atualização dos parâmetros; e suaviza a distribuição de dados de treinamento, a aproximando da distribuição estacionária da política, o que é vital para a estabilidade global do algoritmo foda-da-politica (*off-policy*), o que significa que ele pode aprender a função de valor ótima utilizando dados gerados por políticas de comportamento antigas.

4.2. Métodos Baseados em Política e sua Otimização

A segunda família de algoritmos de Aprendizado por Reforço Profundo (DRL) são os Métodos Baseados em Política (*Policy-Based Methods*, PBMs). Enquanto os métodos baseados em valor (como o DQN) buscam aproximar a função Q(s, a) para então derivar uma política de forma implícita (como ϵ-*greedy*), os PBMs focam em encontrar a estratégia comportamental ótima π^* otimizando diretamente uma política parametrizada π(a|s, θ) no espaço de parâmetros $\theta \in \mathbb{R}^d$.

A política π_θ define o comportamento do agente, mapeando os estados percebidos do ambiente para uma distribuição de probabilidade sobre as ações admissíveis. No contexto do TSP, isso é particularmente vantajoso, pois ao invés de buscar uma função de valor que pode ser descontínua ou difícil de generalizar em grafos complexos, a política define uma distribuição de probabilidade sobre a próxima cidade a ser visitada, dependendo da cidade atual e do histórico da rota parcial.

A adoção desta abordagem direta no lugar da aprendizagem supervisionada clássica é motivada por uma limitação fundamental da Otimização Combinatória: a escassez de dados rotulados. Em uma abordagem supervisionada (como as *Pointers Networks* originais), o treinamento exigia um vasto conjunto de pares contendo instâncias de grafos e suas respectivas soluções ótimas. Contudo, gerar esses rótulos ótimos exige a execução de solucionadores exatos em problemas NP-difíceis (10). O aprendizado por Reforço supera essa barreira ao eliminar a necessidade de um "professor" externo, o agente aprende heurísticas eficazes guiado apenas pelo sinal de recompensa.

Este paradigma, normalmente denominado como Otimização Combinatória Neural, trata a construção da solução como um processo sequencial probabilístico. O objetivo deixa de ser a minimização de um erro em relação a resposta ótima e passa a ser a maximização de um funcional de desempenho $\mathrm{J}(\theta)$, definido como a esperança matemática do retorno sobre todas as sobre todas as trajetórias possíveis induzidas pela política:

$$\mathrm{J}(\theta) = \mathbb{E}_{\tau \sim p_\theta(\tau)}[\boldsymbol{\mathcal{R}}(\tau)] = \sum_{\tau} p_\theta(\tau)\boldsymbol{\mathcal{R}}(\tau)$$

Onde τ representa uma trajetória completa (uma permutação válida de cidades), $\mathcal{R}(\tau)$ é o retorno acumulado (negativo do custo da rota), e $p_\theta(\tau)$ é a probabilidade conjunta da trajetória sob a política parametrizada.

A vantagem teórica desta formulação é que ela simplifica o problema de otimização discreta ao atribuir probabilidades contínuas e diferenciáveis às arestas do grafo, transformando a busca combinatória rígida em um problema de otimização estocástica contínua. Isso permite a aplicação de métodos de Subida de Gradiente (*Gradient Ascent*) para ajustar os parâmetros θ:

$$\theta_{k+1} = \theta_k + \alpha \boldsymbol{\nabla}_\theta \mathrm{J}(\theta_k)$$

O desafio central , que motiva o uso do algoritmo REINFORCE, é que o gradiente $\boldsymbol{\nabla}_\theta \mathrm{J}(\theta)$ não pode ser calculado diretamente dos próprios parâmetros que estão sendo otimizados.

4.2.1. Gradiente de Política: REINFORCE

A abordagem de Gradiente de Política (*Policy Gradient*) constrói o fundamento matemático para a otimização direta de uma política estocástica $\pi(a|s, \theta)$, parametrizada por um vetor θ. Historicamente, o algoritmo REINFORCE, proposto por Williams (1992),

estabeleceu-se como o método canônico de Monte Carlo para estimar os gradientes da função função objetivo $J(\theta)$ em tarefas episódicas.

A base teórica para esta otimização é formalizada pelo Teorema do Gradiente de Política. Para tornar o gradiente computável via amostragem, utiliza-se a identidade $\nabla\pi = \pi\nabla \ln \pi$. Isso permite transformar o gradiente da esperança (que é difícil de calcular diretamente pois a distribuição dos dados muda com θ) na esperança do gradiente, uma forma adequada para estimativa via Monte Carlo que estabelece a direção na qual os parâmetros θ devem ser ajustados para maximizar o retorno esperado. Formalmente, o gradiente do desempenho esperado é dado por:

$$\nabla\theta\, J(\theta) = \mathbb{E}_{\pi}\left[\sum_{t=0}^{T} \nabla_{\theta} ln\, \pi\left(a_t|s_t,\ \theta\right) G_t\right]$$

Onde G_t representa o retorno acumulado a partir do passo t. No contexto do TSP, como o problema é episódico e a recompensa é recebida apenas ao final (o negativo do custo total), G_t corresponde frequentemente ao custo total da trajetória completa τ gerada pela rede.

O termo $\nabla_{\theta} \ln\pi(a_t|s_t, \theta)$, conhecido como *score function*, indica a direção no espaço de parâmetros que aumenta a probabilidade de selecionar a ação a_t. O algoritmo, portanto, opera sob um princípio intuitivo de reforço: se a trajetória gerada resulta em um retorno alto (custo baixo), os parâmetros são ajustados para tornar as ações dessa trajetória mais prováveis no futuro, caso contrário as probabilidades são suprimidas.

Para implementar o teorema na prática, o algoritmo REINFORCE utiliza uma estimativa empírica da esperança matemática. Este mecanismo requer a execução de *roll-outs* completos, ou seja, a simulação de episódios inteiros do TSP até que o ciclo seja fechado. Esta metodologia diverge se diferencia fundamentalmente de abordagens baseadas em Diferença Temporal (TD), como o DQN, pois não utiliza *bootstrapping* (estimativas baseadas em outras estimativas), garantindo que o gradiente seja não-enviesado, embora sujeito a alta variância.

Uma limitação crítica das estimativas de Monte Carlo puras é a alta variância introduzida pela estocasticidade das rotas amostradas e pela magnitude dos retornos G_t. Para mitigar esta instabilidade sem introduzir viés, subtrai-se uma função de linha de base (*baseline*), $b(s_t)$, do retorno. A regra de atualização dos parâmetros torna-se:

$$\theta_{k+1} = \theta_k + \alpha \sum_{t=0}^{T} \nabla_{\theta}\, ln\, \pi\left(a_t|s_t,\ \theta_k\right)\left(G_t - b\left(s_t\right)\right)$$

Para que a subtração de $b(s_t)$ não enviese o gradiente a *baseline* deve ser independente da ação a_t condicionada ao estado. No contexto de Otimização Combinatória Neural aplicada ao TSP, uma escolha eficaz e computacionalmente eficiente para $b(s_t)$ é o custo de uma rota gerada por uma política gulosa determinística (ou a média móvel de rotas passadas). Desta forma, o termo $(G_t - b(s_t))$ atua como uma vantagem, pois o agente aprende apenas se a rota amostrada for melhor do que a rota de referência atual para aquela instância específica do grafo, normalizando o gradiente em relação à dificuldade intrínseca da instância.

Esta técnica mostrou-se eficiente para aproximar o TSP, por exemplo, em trabalhos que utilizam o REINFORCE junto com modelos de atenção, mecanismos de aprendizado que são invariantes ao tamanho da entrada e que levam em conta a estrutura de um grafo por meio de um procedimento de mascaramento, melhorando significativamente o desempenho do TSP em até 100 cidades.

4.2.2. Métodos Ator-Crítico (*Actor-Critic*)

A arquitetura do Ator-Crítico (*Actor-Critic*, AC) unifica as famílias de métodos baseados em valor e em política, buscando mitigar a alta variância ligada aos estimadores de gradiente puros (como o REINFORCE) através da introdução de viés controlado via aproximação de funções. Enquanto o REINFORCE depende de retornos de Monte Carlo completos (G_t) para estimar o gradiente $\nabla J(\theta)$ (o que garante a ausência de viés assintótico, mas sofre com ruído excessivo devido à estocasticidade acumulada ao longo de trajetórias longas) os métodos AC utilizam o conceito de *bootstrapping*.

Nesta abordagem, a estimativa do retorno futuro não é obtida aguardando o término do episódio, mas sim aproximada pela própria função de valor aprendida pelo agente. Matematicamente substitui-se o retorno amostrado G_t por uma estimativa de Diferença Temporal (TD), fundamentada na aplicação do Operador de Bellman. Embora o uso de uma função de valor aproximada $V_\omega(s)$ introduz viés (uma vez que V_ω pode não convergir inicialmente para V^π), esta técnica reduz drasticamente a variância das atualizações, permitindo um aprendizado mais estável e eficiente, além de viabilizar a aplicação em tarefas contínuas onde episódios não terminam naturalmente.

Sob a ótica da Programação Dinâmica os algoritmos Ator-Crítico podem ser interpretados como uma implementação estocástica e aproximada da Iteração de Política Generalizada (*Generalized Policy Iteration*, GPI). Diferentemente do REINFORCE, que apenas otimiza a política, o AC mantém e otimiza dois conjuntos distintos de parâmetros,

decompondo o problema em dois processos iterativos: a Avaliação de Política (estimar V^{π}) e a Melhoria de Política (otimizar π com base em V^{π}). Estes componentes são definidos funcionalmente como:

- O Ator (*Actor*): É a estrutura responsável pela tomada de decisão, paremetrizando explicitamente a política estocástica π(a|s, θ). O objetivo do Ator é ajustar os parâmetros θ para maximizar o funcional de desempenho $\mathcal{J}$(θ) via subida de gradiente. Diferentemente de métodos puramente baseados em valor, o Ator não necessariamente consulta uma função Q gulosa para selecionar ações.

- O Crítico (*Critic*): É a estrutura responsável pela avaliação da política atual, estimando a função de valor V(s; ω) (ou Q(s, a; ω)) parametrizadas por pesos ω. O papel teórico do Crítico é projetar a função de valor real da política $V^{\pi\theta}$ no espaço de funções representáveis pela rede neural, minimizando o erro de predição temporal. O sinal gerado pelo Crítico atua como uma *baseline* de baixa variância para guiar as atualizações do Ator.

A interação entre estes componentes ocorre através do Erro de Diferença Temporal (TD), denotado como δ_t. Este sinal escalar é fundamental pois substitui o retorno completo G_t (utilizado no REINFORCE) por uma estimativa *bootstrapped* de um passo. Matematicamente, o erro TD representa uma amostra da discrepância entre a estimativa atual e a aplicação do Operador de Bellman aproximado:

$$\delta_t(\omega) = r(s_t, a_t) + \gamma V(s_{t+1}; \omega) - V(s_t; \omega)$$

Se $\delta_t > 0$, indica que a transição resultou em um estado melhor do que o esperado pela estimativa atual, sugerindo que a probabilidade da ação a_t deve ser aumentada. Se $\delta_t < 0$, a ação teve um desempenho inferior à expectativa média do estado, devendo ser suprimida.

Para arquiteturas de *Deep* RL, as atualizações dos parâmetros não ocorre em valores tabulares de preferência, mas sim através de descida (Crítico) e subido (Ator) de gradiente. As regras de atualização simultânea, com taxas de aprendizado α_θ e α_ω, são dadas por:

1. Atualização do Crítico (Minimização do Erro Quadrático do TD):

$$\omega_{t+1} = \omega_t + \alpha_\omega \delta_t(\omega_t) \nabla_\omega V(s_t; \omega_t)$$

2. Atualização do Ator (Gradiente de Política com Crítico):

$$\theta_{t+1} = \theta_t + \alpha_\theta \delta_t(\omega_t) \nabla_\theta ln\, \pi(a_t|s_t;\ \theta_t)$$

Nesta formulação o erro TD δ_t atua como uma estimativa da Função de Vantagem ($A^\pi(s, a) = Q^\pi(s, a) - V^\pi(s)$). O uso do Crítico reduz significativamente a variância da estimativa do gradiente em comparação com métodos de Monte Carlo, embora introduza um viés decorrente da aproximação da função de valor $V(s; \omega)$.

Para mitigar o *trade-off* entre viés (de métodos de um passo, TD(0)) e variância (de métodos Monte Carlo), os métodos Ator Crítico podem ser generalizados através de Traços de Elegibilidade (*Eligibility Traces*). Esta técnica unifica as abordagens temporal e estrutural da atribuição de valor.Introduz-se um parâmetro de decaimento $\lambda \in (0, 1)$, que pondera a influência de recompensas futuras.

O algoritmo Ator-Crítico com Traços (λ-AC) mantém vetores de traços e_t^θ e e_t^ω para o Ator e o Crítico, respectivamente. Os traços acumulam os gradientes passados e decaem exponencialmente:

$$e_t^\theta = \gamma\lambda e_{t-1}^\theta + \nabla_\theta ln\, \pi(a_t|s_t,\ \theta_t)$$

A atualização dos parâmetros passa a considerar todo o histórico recente de ativações ponderado pelo erro atual: $\theta_{t+1} = \theta_t + \alpha_\theta \delta_t e_t^\theta$. Isso permite que o algoritmo propague o aprendizado de uma recompensa tardia para ações tomadas muitos passos atrás, acelerando a convergência em problemas como o TSP.

Algoritmos modernos de *Deep* RL, como o A3C (*Asynchronous Advantage Actor-Critic*) e PPO (*Proximal Policy Optimization*), são evoluções diretas desta base teórica, incorporando execução paralela e restrições na região de confiança para estabilizar o processo de otimização não-linear.

5. Análise de Convergência e Estabilidade

A validação teórica dos algoritmos de Aprendizado por Reforço fundamentam-se, normalmente, na análise das propriedades de contração dos operadores de Bellman e na existência de pontos fixos únicos para as equações de otimalidade. No entanto, a transição do paradigma tabular exato para o uso de aproximadores de função parametrizados altera

fundamentalmente a natureza do problema de otimização e as associadas garantias de estabilidade. Ao projetar a função de valor em um subespaço de hipóteses restrito (como o de uma Rede Neural), o objetivo do algoritmo não é mais encontrar o ponto fixo exato, passando a ser a minimização de uma função de erro ou a maximização do desempenho esperado, o que frequentemente introduzem vieses e riscos de divergência não presentes nos métodos exatos. Consequentemente, este capítulo dedica-se a investigar a dinâmica do aprendizado com base na teoria da Aproximação Estocástica, analisando como a interação entre aproximação não-linear, *bootstrapping* e a amostragem impacta a estabilidade dos estimadores e quais mecanismos são necessários para aproximar o comportamento do sistema da convergência desejada.

5.1. Convergência em Métodos Tabulares

Para analisar a convergência dos algoritmos de Aprendizado por Reforço, é fundamental abstrair as atualizações iterativas como operações em um espaço vetorial. No contexto tabular, onde o espaço de estados $\mathscr{S}$ é finito, qualquer função de valor V(s) pode ser representada como um vetor $V \in \mathbb{R}^{|\mathscr{S}|}$. Desta forma, a atualização de Bellman, utilizada tanto em Programação Dinâmica quanto em métodos de Monte Carlo e TD, atua como um operador funcional: T: $\mathbb{R}^{|\mathscr{S}|} \rightarrow \mathbb{R}^{|\mathscr{S}|}$. Para o problema de controle ótimo, define-se o Operador de Otimalidade de Bellman T^* (como apresentado no Capítulo 2.2.1) tal que a busca pela política ótima π^* é matematicamente equivalente à busca pelo ponto fixo V^* que satisfaz a equação $T^*V^* = V^*$.

5.1.1. Propriedades de Contração e o Ponto Fixo de Banach

A garantia teórica de que algoritmos baseados em valor convergem para a solução ótima reside nas propriedades geométricas do operador T^*. Utilizando a norma máxima (ou norma infinito), definida como $||V||_\infty = \max_{s \in \mathscr{S}} |V(s)|$, demonstra-se que o operador de Bellman é uma γ-contração. Isto implica que, para quaisquer dois vetores V_1 e V_2, a aplicação do operador reduz a distância entre eles por um fator γ: $||T^*V_2 - T^*V_1||_\infty \leq \gamma||V_2 - V_1||_\infty$, onde $\gamma \in [0,1)$ é o fator de desconto. Consequentemente, pelo Teorema do Ponto Fixo de Banach, a aplicação iterativa do operador $V_{k+1} = T^*V_k$ garante a convergência para um único ponto fixo V^*, independentemente da inicialização V_0, com uma taxa de convergência geométrica. Esta propriedade assegura a estabilidade incondicional de métodos como *Value Iteration* em domínios onde a representação tabular é viável.

5.1.2. Convergência sob Amostragem Estocástica

Enquanto a Programação Dinâmica assume acesso direto ao modelo de transição para computar a esperança matemática exata no operador de Bellman, algoritmos *model-free* como o *Q-Learning* dependem de amostras estocásticas de transições (s,a,r,s'). Neste cenário a atualização do valor Q(s, a) pode ser interpretada como uma aproximação com ruído do operador de contração. A convergência para Q^* é preservada, com probabilidade 1, desde que todos os pares estado-ação sejam visitados infinitas vezes e que a sequência de taxas de aprendizado $\{\alpha_t\}$ satisfaçam as condições de Robbins-Monro:

$$\sum_{t=0}^{\infty} \alpha_t = \infty \text{ e } \sum_{t=0}^{\infty} \alpha_t^2 < \infty$$

A primeira condição assegura que os pesos são grandes o suficiente para superar diferentes condições iniciais diversas, enquanto a segunda garante que a oscilação induzida pelo ruído estocástico da amostragem seja amortecida ao longo do tempo, permitindo que o estimador estabilize no ponto fixo teórico.

5.2. Aprendizado com Aproximação de Função

A transição da teoria tabular para a aplicação prática em Otimização Combinatória impõe uma mudança fundamental na natureza do problema de aprendizado. Conforme discutido no Capítulo 3, a cardinalidade do espaço de estados do TSP inviabiliza a representação direta da função de valor V(s). Ao adotarmos um aproximador de função parametrizado V(s; ω) (nesse caso, uma Rede Neural Profunda com pesos $\omega \in \mathbb{R}^d$) restringimos a busca pela solução a um subespaço de hipóteses H de dimensão muito inferior à do espaço original ($d \ll |\mathcal{S}|$). Consequentemente, o objetivo do algoritmo deixa de ser identificar o ponto fixo (que muito provavelmente não reside em H) e passa a ser a determinação do conjunto de parâmetros ω* que minimiza a distância entre a função aproximada e a função real, com base em um erro médio ponderado pela frequência que cada estado é visitado.

5.2.1. A geometria da Atualização

Ao analisar cada iteração de um algoritmo baseada em valor com aproximação de funções, ela pode ser interpretada como um processo de duas etapas. Primeiramente,

aplica-se o Operador de Bellman T à estimativa atual $V(s; \omega_k)$, o que gera uma nova função alvo que, em geral, pode situar-se fora do subespaço representável pela rede neural. Em seguida, para tornar a atualização viável, é necessário projetar a função alvo de volta ao espaço de parâmetros H. Define-se portanto, o Operador de Projeção Π, que mapeia a função de valor arbitrária para sua melhor aproximação dentro da arquitetura da rede. A dinâmica do aprendizado passa a ser governada pela composição destes operadores, buscando resolver a Equação de Bellman Projetada (PBE): $\phi\omega = \Pi T(\phi\omega)$, onde ϕ representa a matriz de características ou a transformação não linear realizada pelas camadas da rede.

5.2.2. Instabilidade e Perda da Propriedade de Contração

A introdução do operador de projeção Π acarreta uma perda crítica das garantias de convergência observadas nos métodos tabulares. Embora o Operador de Bellman T seja uma γ-contração na norma infinito ($\| \cdot \|_\infty$), a operação composta ΠT não preserva necessariamente esta propriedade. Em arquiteturas não-lineares ou quando a distribuição de amostragem dos estados não coincide com a distribuição estacionária da política (como ocorre no uso de *Replay Buffers*), a projeção pode expandir o erro em certas direções do espaço de parâmetros. Geometricamente, isso significa que a atualização iterativa pode fazer com que a estimativa de valor oscile ou até divirja indefinidamente, em vez de convergir suavemente para um ponto fixo, invalidando a aplicação direta do Teorema do Ponto Fixo de Banach neste contexto.

5.2.3. A “Tríade Mortal” do *Deep Q-Learning*

Esta instabilidade estrutural é sistematizada sob o conceito da “Tríade Mortal” (*The Deadly Triad*), que postula que o risco de divergência em algoritmos de Aprendizado por Reforço surge da combinação simultânea de três elementos:

1. Aproximação de Função, que introduz o erro de projeção e a impossibilidade de representação exata;

2. *Bootstrapping*, utilizado em métodos de Diferença Temporal como o DQN, onde a estimativa atual depende de estimativas futuras, propagando e potencialmente amplificando viesses; e

3. Treinamento Fora-da-Política (*Off-Policy*), onde a distribuição de dados de treinamento difere da política que está sendo avaliada.

O algoritmo DQN, ao empregar redes neurais, alvos TD e *Experience Replay*, incorpora os três elementos da tríade, tornando a análise de sua estabilidade dependente não de garantias de contração simples, mas de mecanismos mitigatórios como Target Networks, cuja função teórica é estabilizar o alvo da projeção temporariamente, aproximando o problema de uma regressão supervisionada estacionária.

5.3. Intuição via o Método da Equação Diferencial Ordinária (Ordinary Differential Equation ,ODE)

Diante da impossibilidade de garantir a contração global do operador de Bellman projetado (ΠT) em arquiteturas de redes neurais profundas, a análise de convergência deve transitar da busca por pontos fixos em espaços de Banach para a teoria da Aproximação Estocástica. Nesta perspectiva, o processo de treinamento do agente não é visualizado como uma sequência de contrações geométricas, mas sim como a integração numérica ruidosa de um sistema dinâmico contínuo. A intuição central do Método da ODE postula que, conforme a taxa de aprendizado α tende a zero, a sequência de parâmetros discretos $\{\omega_k\}$ gerada pelo algoritmo (seja DQN ou REINFORCE) interpola assintoticamente a trajetória de uma Equação Diferencial Ordinária determinística, cujos pontos de equilíbrio correspondem aos ótimos locais da função objetivo ou aos pontos fixos da política.

5.3.1. Formalização da Atualização como Equação de Diferenças

Podemos formalizar a regra de atualização dos parâmetros ω da rede neural na iteração k através de uma equação de diferenças geral:

$$\omega_{k+1} = \omega_k + \alpha_k Y_k$$

Onde Y_k é a atualização baseada na amostra obtida. Para analisar a convergência, decompomos Y_k em duas componentes: o comportamento médio esperado (o “sinal”) e o ruído de amostragem. Seja $h(\omega_k) = \mathbb{E}[Y_k|\omega_k]$ o campo vetorial médio, que representa a direção “verdadeira” de otimização se tivéssemos acesso a infinitas amostras. Desta forma podemos reescrever a atualização como:

$\omega_{k+1} = \omega_k + \alpha_k [h(\omega_k) + M_{k+1}]$

Nesta formulação:

1. $h(\omega_k)$ (Campo Médio): Representa a dinâmica determinística do sistema. No caso do REINFORCE, $h(\omega) = \nabla J(\omega)$, o gradiente da performance esperada. No DQN, $h(\omega)$ representa o erro de Bellman esperado sobre a distribuição de transições.

2. M_{k+1} (Ruído): Definido como $M_{k+1} = Y_k - h(\omega_k)$. Este termo captura a estocasticidade ligada ao TSP, nesta pesquisa a variação na recompensa causada pela seleção de uma instância do grafo ou pela política estocástica $\pi(a| s)$.

Crucialmente, por construção, M_{k+1} é uma diferença de martingale, ou seja, $\mathbb{E}[M_{k+1}|\omega_k] = \mathbb{E}[Y_k - h(\omega_k)|\omega_k] = 0$. O objetivo da análise de convergência torna-se, portanto, demonstrar que, sob certas condições da taxa de aprendizado α_k, o efeito acumulado do ruído M_{k+1} desaparece assintoticamente, permitindo que a trajetória dos pesos siga a direção ditada por $h(\omega_k)$.

5.3.2. A Trajetória Limite e o Campo Vetorial Médio

O Método da Equação Diferencial Ordinária (EDO) propõe uma mudança de perspectiva, em vez de analisar a recursão discreta e ruidosa diretamente, analisamos o comportamento do sistema no limite contínuo. O princípio fundamental é que conforme a taxa de aprendizado α_k tende a zero, as atualizações dos parâmetros tornam-se infinitesimais. Se interpolarmos os valores discretos ω_k ao longo de uma escala de tempo contínua t, a sequência de parâmetros aproxima-se da trajetória solução da seguinte EDO determinística:

$\omega^{\bullet}(t) = h(\omega))$

Esta equação descreve o fluxo do sistema na ausência de ruído. Para o problema do TSP, a estabilidade do algoritmo de aprendizado depende das propriedades deste campo vetorial $h(\omega)$:

- Em Métodos de Política (REINFORCE): Como $h(\omega)$ é o gradiente de uma função objetivo ($h = \nabla J$), a EDO descreve um sistema de subida de gradiente contínua. A

teoria de sistemas dinâmicos garante que tais trajetórias convergem para pontos de equilíbrio estáveis (máximos locais de J), desde que a função seja suave e limitada. Isso sugere que o REINFORCE, apesar da alta variância, é teoricamente estável.

- Em Métodos de Valor (DQN): A situação é mais complexa devido à "Tríade Mortal" discutida no Capítulo 5.2.3. O campo h(ω) do DQN não é necessariamente o gradiente de uma função de erro fixa (devido ao *bootstrapping*). Consequentemente, a EDO associada pode não ter pontos de equilíbrio estáveis (atratores), podendo divergir.

A aplicação prática dessa teoria no projeto está na interpretação da estabilidade assintótica. Para que o algoritmo de *Deep* RL convirja para uma solução útil (uma heurística de roteamento eficaz), é necessário que a trajetória da EDO permaneça dentro da região de estabilidade de um atrator estável por tempo suficiente para que o ruído estocástico seja amortecido.

5.3.3. Condições de Robbins-Monro

A validade da aproximação pela EDO e a garantia que o ruído M_{k+1} não desestabilize o aprendizado dependem estritamente do escalonamento da taxa de aprendizado α_k. A teoria clássica de Aproximação Estocástica estabelece as condições de Robbins-Monro para a sequência $\{\alpha_k\}$:

1. Suficiência para Aprendizado ($\sum_{k=0}^{\infty} \alpha_k = \infty$):

   Esta condição exige que a série de passos seja divergente. Intuitivamente, isso garante que o algoritmo nunca pare prematuramente. Mesmo que a taxa de aprendizado decaia, ela deve permitir que os parâmetros ω percorrem uma distância arbitrariamente grande no espaço de parâmetros. Isso é vital no início do treinamento do TSP, onde a inicialização aleatória da rede neural pode colocar o agente em uma região de performance muito baixa, exigindo grandes ajustes para alcançar a região de uma política ótima.

2. Supressão de Ruído ($\sum_{k=0}^{\infty} \alpha_k^2 < \infty$):

   Esta condição exige que o quadrado dos passos seja convergente. Ela está

diretamente ligada à variância do estimador. No contexto do TSP o gradiente estocástico tem alta variância (pois cada grafo gera uma rota diferente). Se α_k não diminuir rápido o suficiente (respeitando esta condição quadrática), a variância trazida pelo ruído M_{k+1} manterá os pesos oscilando continuamente em torno da solução ótima, sem nunca estabilizar (convergir). A condição garante que o peso do ruído decaia assintoticamente a zero.

Embora a teoria exija que $\alpha_k \rightarrow 0$, na implementação prática em redes neurais profundas frequentemente violam a segunda condição usando uma taxa de aprendizado constante ou com decaimento lento. Sob a ótica da EDO isso implica que o algoritmo não converge para um ponto fixo ω^*, mas sim para uma distribuição estacionária em torno de ω^*. Visando a generalização no TSP, isso é frequentemente aceitável e até desejável, pois permite que o agente continue adaptando-se a novas instâncias de grafos em um ambiente que, na prática, se comporta como não estacionário.

6. Considerações Finais e Próximos Passos

6.1. Síntese

A investigação realizada permitiu estruturar uma base teórica que conecta a a Otimização Combinatória clássica aos modernos algoritmos de Aprendizado por Reforço Profundo. A análise demonstrou que a resolução do Problema do Caixeiro Viajante (TSP) sob a ótica de *Learning to Optimize* não é apenas uma substituição de heurísticas manuais por redes neurais, mas uma mudança fundamental na natureza matemática do problema de otimização, com profundas implicações na garantia de estabilidade das soluções.

Foi estabelecido que, embora o TSP possua um modelo determinístico conhecido o crescimento fatorial do espaço de estados inviabiliza a aplicação de Operadores de Bellman exatos (métodos tabulares), cujas garantias de convergência estão no Teorema do Ponto Fixo de Banach e na norma infinito. A formalização do problema como um Problema de Caminho Mais Curto Estocástico (SSP) permitiu redefinir o objetivo, em vez de buscar a otimalidade global de uma instância isolada via varredura exaustiva, busca-se a convergência assintótica de uma política parametrizada capaz de generalizar sobre a distribuição de instâncias.

A pesquisa evidenciou que a estabilidade do aprendizado depende criticamente da geometria do espaço de hipóteses escolhido para a aproximação. A análise das propriedades de simetria do TSP demonstrou que aproximadores genéricos (como MLPs) introduzem ineficiências amostrais que dificultam a convergência. Concluiu-se que a adoção de Viés Indutivo Geométrico, através de *Graph Neural Networks* (GNNs) e Mecanismos de Atenção, é crucial para garantir as propriedades de invariância e equivariância à permutação. Adicionalmente, o mecanismo de mascaramento (*masking*) provou-se ser a solução estrutural necessária para restringir o espaço de ações estocásticas ao subespaço de soluções viáveis (permutação), eliminando a necessidade do algoritmo aprender restrições rígidas via penalização de recompensa.

A análise teórica identificou que a introdução de aproximadores de funções rompe a propriedade de contração monótona do Operador de Bellman. Ao projetar a função de valor em um subespaço de parâmetros de baixa dimensão, a operação composta (Bellman Seguido de Projeção) pode expandir o erro, levando a fenômenos de divergência. Esta instabilidade foi caracterizada formalmente pela “Tríade Mortal”, composta pela interação perigosa entre: Aproximação e função, *bootstrapping* (estimativas baseadas em estimativas) e amostragem fora e política (*off-policy*). Esta constatação teórica fundamenta a distinção entre os métodos que são por natureza mais instáveis devido ao *bootstrapping* (como DQN)

e métodos que embora apresentem maior variância evitam a tríade ao otimizar diretamente a recompensa (como REINFORCE).

Por fim, diante da impossibilidade de garantir a convergência via ponto fixo em arquiteturas profundas, a pesquisa recorreu à teoria da Aproximação Estocástica e ao Método da Equação Diferencial Ordinária (EDO) para caracterizar a dinâmica de aprendizado. Conclui-se que o processo de otimização dos parâmetros da rede neural pode ser interpretado como a discretização ruidosa de um sistema dinâmico contínuo. A estabilidade do algoritmo não reside na eliminação do erro de aproximação, mas na garantia de que a trajetória dos parâmetros permaneça na bacia de atração de um ponto de equilíbrio estável. Esta análise justifica teoricamente a necessidade de estratégias de ajuste da taxa de aprendizado que satisfaçam as condições de Robbins-Monro, assegurando o amortecimento do ruído estocástico ligado à amostragem de grafos e trajetórias.

6.2. Cronograma Futuro

7. Referências Bibliográficas

[1] Arora, S., and Barak, B. Computational complexity: A modern approach. Cambridge University Press, Cambridge, 2009.

[2] Bengio, Y., Lodi, A., and Prouvost, A. Machine learning for combinatorial optimization: A methodological tour d'horizon. European Journal of Operational Research 290, 2 (2021), 405–421.

[3] Cappart, Q., Chételat, D., Khalil, E. B., Lodi, A., Morris, C., and Veličković, P. Combinatorial optimization and reasoning with graph neural networks. Journal of Machine Learning Research 24, 130 (2023), 1–61.

[4] Garey, M. R., and Johnson, D. S. Computers and intractability: A guide to the theory of NPcompleteness. W. H. Freeman and Co., San Francisco, Calif., 1979. A Series of Books in the Mathematical Sciences.

[5] Mazyavkina, N., Sviridov, S., Ivanov, S., and Burnaev, E. Reinforcement learning for combinatorial optimization: A survey. Computers & Operations Research 134 (2021), 105400.

[6] Mannor, Shie and Mansour, Yishay and Tamar, Aviv. Reinforcement Learning: Foundations. Cambridge University Press, 2026. Disponível em: https://sites.google.com/view/rlfoundations/home. Acesso em: 31 jan. 2026.

[7] Powell, W. Reinforcement Learning and Stochastic Optimization: A Unified Framework for Sequential Decisions. Wiley, 2022.

[8] Sutton, R., and Barto, A. Reinforcement Learning, second edition: An Introduction. Adaptive Computation and Machine Learning series. MIT Press, 2018.

[9] Yang, Y., and Whinston, A. B. A survey on reinforcement learning for combinatorial optimization. CoRR abs/2008.12248 (2020).

[10] YAO, Shunyu; LIN, Xi; WANG, Jiashu; ZHANG, Qingfu; WANG, Zhenkun. Rethinking Supervised Learning-Based Neural Combinatorial Optimization for Routing Problem. ACM Transactions on Evolutionary Learning, Volume 5, Issue 4, p. 1-24, 6 dez. 2025. Disponível em: https://doi.org/10.1145/3694690. Acesso em: 31 jan. 2026.